

به نام خداوند جان و خرد

گزارش کار جلسه اول آزمایشگاه طراحی سیستم‌های دیجیتال

اعضای گروه

پوریا رحمانی ۴۰۲۱۱۱۴۱۸

امیرمحمد شربتی ۴۰۲۱۰۶۱۱۲

توجه: از آنجا که کاری که در آزمایشگاه کردیم از طراحی‌های شماتیک که برای پیش‌گزارش آماده کردیم مجزا بود، تنها همین فایل را برای گزارش کار آپلود خواهیم کرد!

بخش‌های آزمایش

- طراحی یک شمارنده صعودی چهار بیتی (بعد از ۱۵ دوباره به صفر برگردد) با کلاک دستی
- طراحی یک شمارنده چهار بیتی با یک بیت کنترل که اگر صفر بود به ترتیب صعودی و اگر یک بود به ترتیب gray counter بشمارد با کلاک دستی
- وصل کردن طراحی بخش پیش به کلاک برد با فرکانس 50MHz

بخش اول

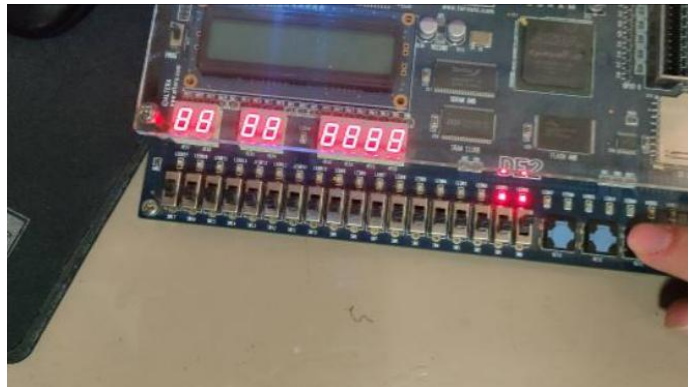
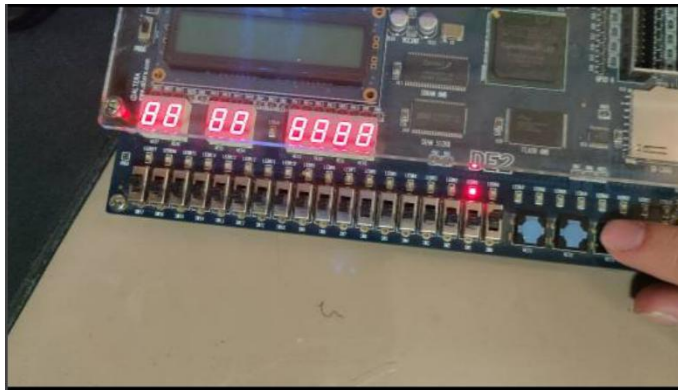
شمارنده را با یک reset که active low است طراحی کردیم. کد verilog نوشته شده را در تصویر زیر می‌بینید (عکس کد نوشته شده از مانیتور در آزمایشگاه بی‌کیفیت بود و در نتیجه از روی آن نوشته و عکس گرفتیم):

```
1 module fourBitCounter(input clk, input reset, output reg[3:0] q);
2   always @(posedge clk, negedge reset) begin
3     if(!reset) begin
4       q <= 0;
5     end else begin
6       q <= q + 1;
7     end
8   end
9 endmodule
```

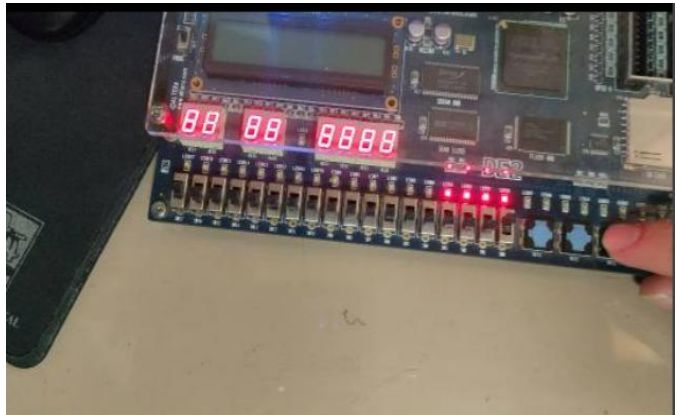
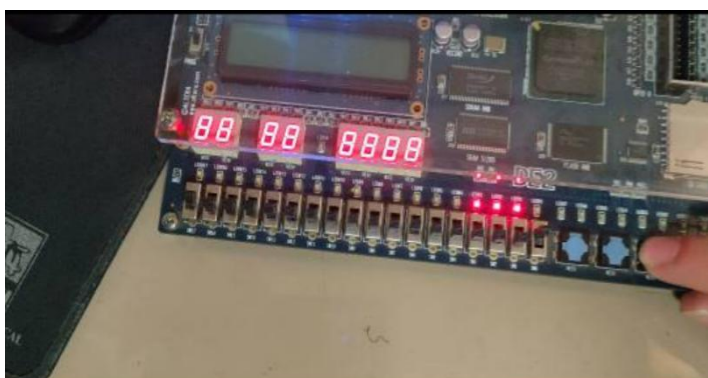
کد بسیار ساده است و شرح خاصی ندارد، صرفاً با هر بار clock (یا در لبه صفر شدن reset) اگر برابر صفر بود شمارنده را برابر صفر می‌گذاریم و اگر برابر یک بود، یکی به مقدار شمارنده اضافه می‌کنیم.

سپس پین‌هایی به ورودی‌ها و خروجی‌ها در بخش pin planner اختصاص دادیم، کد را مجدداً کامپایل کرده و نتایج را روی برد دیدیم.

پین‌های LEDR[3] تا LEDR[0] (به ترتیب پین‌های AE23، AF23، AB21، AC22) را برای خروجی‌ها، KEY[1] (پین N23) را برای کلید کلاک و sw[0] (پین N25) را به reset اختصاص دادیم. تصاویر نتیجه آزمایش را در زیر می‌بینید:



تصاویر بالا مربوط به شمارش از یک تا سه و تصویر پایین مربوط به شمارش از اعداد 13 تا 15 است.



برای توضیح این بخش، ابتدا نیاز است در مورد gray code بیشتر توضیح دهیم. شیوه ساختن ترتیب اعداد gray code n بیتی به صورت بازگشتی به این صورت است که یکبار ترتیب n-1 بیتی gray code را کپی کرده و به سمت چپ آن‌ها صفر اضافه می‌کنیم و پس از آن یک کپی از ترتیب عکس gray code n-1 بیتی که سمت چپ همه آن‌ها یک

بیت یک اضافه شده است را به انتهای list قبلی append کنیم. به راحتی و با استقراء روی تعداد بیت‌ها می‌توان نشان داد که i امین عدد در ترتیب gray counter، برابر $i \oplus (i \gg 1)$ است. بنابراین اگر بخواهیم ببینیم یک عدد 4 بیتی مثل g چندمین عدد در ترتیب gray counter است، کافیت این عبارت را حساب کنیم:

$$g \oplus (g \gg 1) \oplus (g \gg 2) \oplus (g \gg 3)$$

زیرا اگر فرض کنید g ، عدد j ام در ترتیب gray counter باشد، خواهیم داشت:

$$\begin{aligned} &g \oplus (g \gg 1) \oplus (g \gg 2) \oplus (g \gg 3) \\ &= (j \oplus (j \gg 1)) \oplus ((j \gg 1) \oplus (j \gg 2)) \oplus ((j \gg 2) \oplus (j \gg 3)) \oplus ((j \gg 3) \oplus (j \gg 4)) \end{aligned}$$

از آنجا که حاصل xor هر عددی با خودش برابر صفر است و همینطور j یک عدد چهار بیتی است و در نتیجه $4 \gg j = 0$ حاصل عبارت بالا برابر خواهد بود با $0 \oplus 0 \oplus 0 \oplus 0$ که از آنجا که حاصل xor هر عددی با صفر خودش است، حاصل عبارت بالا برابر خود j خواهد بود، همان چیزی که می‌خواستیم!

بنابراین، اگر بخواهیم از یک عدد چهار بیتی مثل g ، عدد بعدی آن را در ترتیب gray counter بدست بیاوریم، کافیت به روش بالا، ابتدا بدست بیاوریم g چندمین عدد در ترتیب gray counter بوده (فرض کنید بدست بیاید که j امین عدد بوده است) و سپس طبق رابطه‌ای که در ابتدای بخش دوم گفتیم عدد $j+1$ ام gray counter را محاسبه کنیم $(j+1) \oplus ((j+1) \gg 1)$. بر همین منطق، کد را پیاده‌سازی کردیم.

تصویر کد پیاده‌سازی شده را در زیر می‌بینید:

```

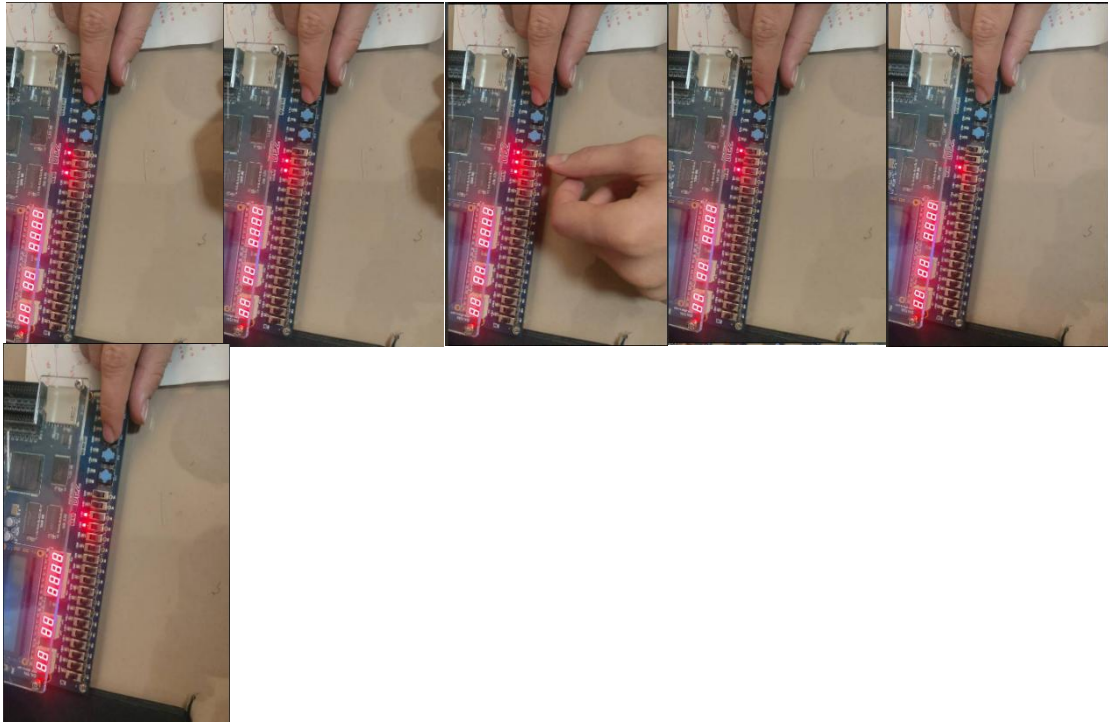
1  module fourBitCounter(input clk, input reset, input ctrl, output reg[3:0] q);
2  reg[3:0] holder;
3  always @(posedge clk, negedge reset) begin
4  |   if(!reset) begin
5  |       q <= 0;
6  |   end else begin
7  |       if(ctrl) begin
8  |           holder = q ^ (q>>1) ^ (q>>2) ^ (q>>3);
9  |           holder = holder + 1;
10 |           q <= holder ^ (holder >> 1);
11 |       end else begin
12 |           q <= q + 1;
13 |       end
14 |   end
15 end
16 endmodule

```

تفاوت این کد با کد بخش قبل دقیقاً در پیاده‌سازی منطق توضیح داده شده است که برای آن از یک reg چهار بیتی holder برای بدست آوردن اندیس عدد در ترتیب gray scale استفاده کرده‌ایم و اینکه شمارنده به صورت صعودی بشمارد یا به ترتیب gray counter هم بر اساس یک سیگنال کنترلی به نام ctrl تعیین می‌شود (اگر این بیت یک باشد، مطابق gray counter می‌شماریم)

پین‌های اختصاص داده شده هم مشابه بخش پیش بودند، صرفاً برای سیگنال ctrl از **sw[1]** (پین N26) استفاده کردیم. تصاویر زیر، نتایج روی برد در آزمایشگاه را نشان می‌دهند.

در تصاویر صفحه بعد در ابتدا شمارش از ۵ تا ۷ به صورت صعودی و سپس طی شدن ترتیب gray counter به صورت 4 -> 5 -> 7 را می‌بینید:



بخش سوم

در این بخش باید به جای کلاک دستی، برای کلاک، از کلاک خود برد استفاده می‌کردیم. برد، یک کلاک 27 مگاهرتزی و یک کلاک 50 مگاهرتزی دارد که دسترسی به کلاک 50 مگاهرتزی از طریق پین **N2** ممکن است. بنابراین باید ورودی کلاک را به جای KEY[1] (یا معادلا پین **N23**) باید به پین **N2** وصل می‌کردیم. تنها نکته‌ی اضافی این بود که از آنجا که کلاک بسیار سریع است و شمارش قابل رویت نمی‌شد، باید با یک reg کمکی تعداد کلاک‌ها را می‌شمریم و هرگاه شماره کلاک به صفر برمی‌گشت، یک بار عملیات مد نظر در کد بخش پیش را انجام می‌دادیم (50 میلیون تقریباً 2^{26} است و ما رجیستر خود را 28 بیتی در نظر گرفتیم. در این صورت حدودا هر 5 ثانیه باشد عدد شمارنده تغییر کند).

کدی که در آزمایشگاه نوشتیم در تصویر پایین قابل مشاهده است. متاسفانه این کد درست عمل نکرد (کامپایل شد ولی عمل شمارش روی برد انجام نشد) منتهی به دلیل کمبود وقت نتوانستیم کد را اصلاح کرده و نتیجه مطلوب را ببینیم ☹️

```

1 module FourBitCounter (input clk, input reset, input ctrl, output reg [3:0] q);
2   reg [3:0] holder;
3   reg [27:0] clk_counter = 0;
4   always @(posedge clk, negedge reset) begin
5     clk_counter <= clk_counter + 1;
6     if (~reset) begin
7       q <= 0;
8       clk_counter <= 0;
9     end else begin
10      if (clk_counter == 0) begin
11        holder = q ^ (q >> 1) ^ (q >> 2) ^ (q >> 3);
12        holder = holder + 1;
13        q <= holder ^ (holder >> 1);
14      end else begin
15        q <= q + 1;
16      end
17    end
18  end
19 end
20 end
21 endmodule
  
```


اما بعد از کلاس کلی وقت گذاشتیم تا بفهمیم مشکل این نسخه از کد چه بوده و چطور میتوانستیم بهتر متوجه این شویم که ایراد کجاست.

اول اینکه چه چیزی نسبت به نسخه پیش تغییر کرد؟

1. استفاده از کلاک اتوماتیک خود مدار که روی فرکانس 50MHz است و در نتیجه تغییر یکی از پین های ورودی

2. استفاده از clk_counter برای اینکه خروجی مدار و شمارش اعداد قابل رویت باشد.

پس شاید بهتر بود ابتدا بدون استفاده از counter کلاک رو تست کنیم. اصلا ببینیم آیا پینی که برای کلاک در نظر گرفتیم درست کار میکند و مطابق انتظار پیش میرود یا نه. یعنی همان نسخه قبلی که با کلاک دستی درست کار میکرد را با کلاک اتوماتیک تست کنیم تا ببینیم clock درست کار میکند و پین به درستی به ورودی منتسب شده است یا نه. (درست است که نمیشد شمارش درست را دید ولی خب تغییر LED ها بیانگر کارکرد درست کلاک و انتساب درست پین ورودی بود)

مورد دیگر (که اگر وقت بود) میشد با کلاک 27MHz مدار را تست کرد تا ببینیم این مدار درست کار میکند یا خیر.

اما خب فرصت نشد تا این کار ها را برای تست مدار انجام دهیم.

اما در مورد خود کد وریلاگی که پیاده سازی شد چه؟

یک نکته که خیلی جالب است این است که برای سنتز درست و پیاده سازی صحیح مدار روی برد لازم است که بلوک های if حتما یک else هم داشته باشند. یعنی مدار باید بداند که اگر این شرط برقرار نبود چه کند! اما در اینجا ما از if(clk_counter == 0) استفاده کردیم در حالی که مدار ممکن است وقتی این شرط درست نیست، رفتار صحیحی نداشته باشد چون چیزی در کد در غیر این صورت تعریف نشده. (در واقع این قاعده بر خلاف زبان های برنامه نویسی مثل C و جاوا، برای سنتز درست تر در HDL لازم است)

مثلا یک راه حل برای حل این مشکل پیاده سازی کد به روشی کمی متفاوت است (برای کارکرد clk_counter):

- از دو بلوک always استفاده کنیم. با هر بار posedge clk ، clk_counter را یکی اضافه کنیم.
- حال در یک بلوک always دیگر شرط (negedge reset, posedge clk_counter[27]) را بگذاریم. در این صورت نیازی به if (clk_counter == 0) نداریم و مشکل قبلی هم از بین میرود (یعنی همان if بدون else).

یعنی کد را بدین شکل تغییر میدادیم:

```
1 module fourBitCounter(input clk, input reset, input ctrl, output reg [3:0] q);
2     reg[3:0] holder;
3     reg[27:0] clk_counter = 0;
4     always @(posedge clk) begin
5         clk_counter <= clk_counter + 1;
6     end
7     always @(negedge reset, posedge clk_counter[27]) begin
8         if(!reset) begin
9             q <= 0;
10        end else begin
11            if (ctrl) begin
12                holder = q ^ (q >> 1) ^ (q >> 2) ^ (q >> 3);
13                holder = holder + 1;
14                q <= holder ^ (holder >> 1);
15            end else begin
16                q <= q + 1;
17            end
18        end
19    end
20 endmodule
```

ممکن است با این تغییرات مدار روی برد به درستی کار کند.